

2. YOLO 모델 구조 및 원리 학습

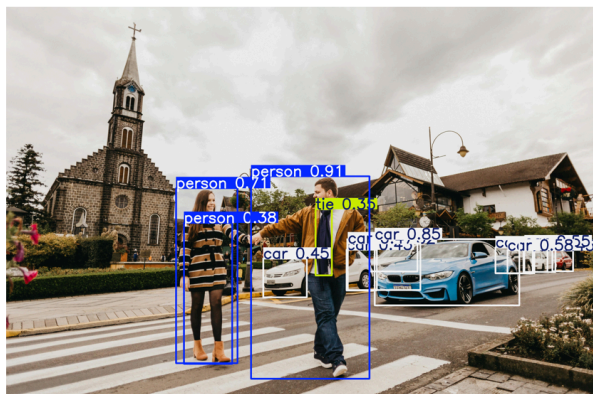
○ 학습목표

- YOLO 객체인식의 원리, 간단히 이해
- YOLO 이용한 객체인식, 용어 이해, 성능평가지표 이해

1 YOLO 미리보기

📌 올로 미리보기(이미지 객체인식)

```
# 설치: pip install ultralytics  
  
from ultralytics import YOLO  
import cv2  
  
# 1. 모델 로드 (YOLOv8)  
# n=nano(제일 가벼움), s, m, l, x 가능  
model = YOLO('yolov8n.pt')  
  
# 2. 이미지에서 탐지  
results = model('test.jpg') # 이미지 경로  
  
# 3. 결과 보기  
results[0].show() # 화면에 표시  
results[0].save('result.jpg') # 저장
```



📌 오픈로 미리보기(동영상 객체인식)

```

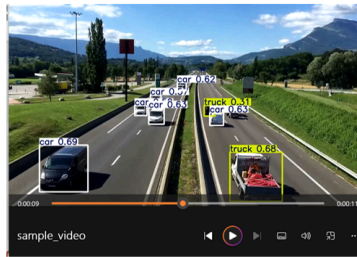
1 from ultralytics import YOLO
2
3 # 모델 로드
4 model = YOLO('yolov8n.pt')
5
6 # 동영상 처리 & 저장
7 results = model('/content/sample_video.mp4', save=True)
8
9 # 또는 화면에 바로 보기
10 # results = model('/content/sample_video.mp4', show=True)

```

Video 1/1 (frame 3/601) /content/sample_video.mp4: 384x640 1 car, 142.3ms
 Video 1/1 (frame 4/601) /content/sample_video.mp4: 384x640 1 car, 140.2ms
 Video 1/1 (frame 5/601) /content/sample_video.mp4: 384x640 2 cars, 145.1ms
 Video 1/1 (frame 6/601) /content/sample_video.mp4: 384x640 2 cars, 215.8ms
 Video 1/1 (frame 7/601) /content/sample_video.mp4: 384x640 2 cars, 236.6ms
 Video 1/1 (frame 8/601) /content/sample_video.mp4: 384x640 2 cars, 219.8ms
 Video 1/1 (frame 9/601) /content/sample_video.mp4: 384x640 2 cars, 218.7ms
 Video 1/1 (frame 10/601) /content/sample_video.mp4: 384x640 2 cars, 221.3ms
 Video 1/1 (frame 11/601) /content/sample_video.mp4: 384x640 2 cars, 226.4ms
 Video 1/1 (frame 6007601) /content/sample_video.mp4: 384x640 3 cars, 138.6ms
 Video 1/1 (frame 601/601) /content/sample_video.mp4: 384x640 3 cars, 145.9ms
 Speed: 3.6ms preprocess, 158.7ms inference, 1.3ms postprocess per image at shape (1, 3)
 Results saved to /content/runs/detect/predict

sample_video.mp4 속성

속성	값
설명	길이 20* 프레임30=600
태그	
설명	비디오
길이	00:00:20
프레임 너비	1280
프레임 높이	720
데이터 속도	3641kbps
총 비트 전송률	3771kbps
프레임 속도	30.00 프레임/초
포맷	
비트 전송률	130kbps
채널	2(스테레오)
오디오 샘플 속도	48.000kHz
미디어	
참여 음악가	

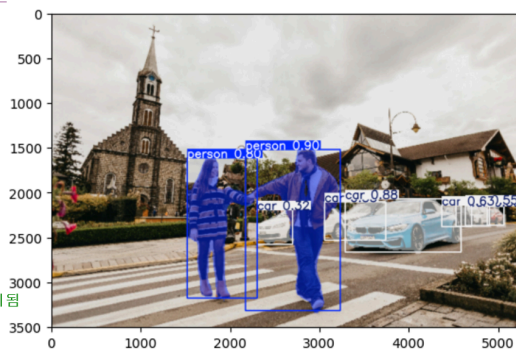


📌 옴로 라벨

names: {0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire hydrant', 11: 'stop sign', 12: 'parking meter', 13: 'bench', 14: 'bird', 15: 'cat', 16: 'dog', 17: 'horse', 18: 'sheep', 19: 'cow', 20: 'elephant', 21: 'bear', 22: 'zebra', 23: 'giraffe', 24: 'backpack', 25: 'umbrella', 26: 'handbag', 27: 'tie', 28: 'suitcase', 29: 'frisbee', 30: 'skis', 31: 'snowboard', 32: 'sports ball', 33: 'kite', 34: 'baseball bat', 35: 'baseball glove', 36: 'skateboard', 37: 'surfboard', 38: 'tennis racket', 39: 'bottle', 40: 'wine glass', 41: 'cup', 42: 'fork', 43: 'knife', 44: 'spoon', 45: 'bowl', 46: 'banana', 47: 'apple', 48: 'sandwich', 49: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog', 53: 'pizza', 54: 'donut', 55: 'cake', 56: 'chair', 57: 'couch', 58: 'potted plant', 59: 'bed', 60: 'dining table', 61: 'toilet', 62: 'tv', 63: 'laptop', 64: 'mouse', 65: 'remote', 66: 'keyboard', 67: 'cell phone', 68: 'microwave', 69: 'oven', 70: 'toaster', 71: 'sink', 72: 'refrigerator', 73: 'book', 74: 'clock', 75: 'vase', 76: 'scissors', 77: 'teddy bear', 78: 'hair drier', 79: 'toothbrush'}

-옴로8부터 가능한 세그멘테이션

```
1 from ultralytics import YOLO
2 import cv2
3
4 # 1. 세그멘테이션 모델 불러오기 (사전학습된 yolov8n-seg)
5 model = YOLO("yolov8n-seg.pt")
6
7 # 2. 이미지 불러오기
8 img = cv2.imread("test.jpg")
9
10 # 3. 추론 실행
11 results = model(img)
12
13 # 4. 마스크 & 박스 그리기
14 annotated = results[0].plot() # 자동으로 mask, box, label 표시됨
15
16 # 5. 결과 저장/출력
17
18 import matplotlib.pyplot as plt
19 plt.imshow(cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB))
```



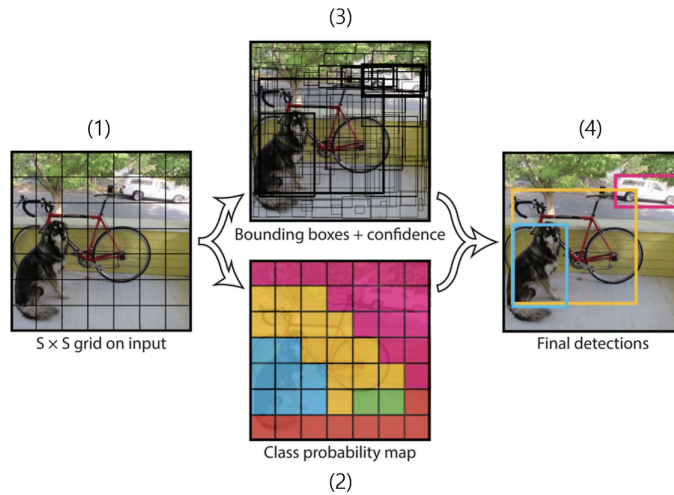
2 YOLO 개념 및 구조

YOLO(You Only Look Once)는 한 번의 forward pass로 이미지 전체에서 object detection을 수행하는 실시간 객체 탐지 모델입니다.

전통적인 객체 검출 방법들(R-CNN, Fast R-CNN)은 2단계 접근법을 사용했습니다. 먼저 관심 영역(Region Proposal)을 찾고, 각 영역에 대해 분류를 수행하는 방식이었죠. 반면 YOLO는 단일 신경망이 전체 이미지에서 바운딩 박스와 클래스 확률을 직접 예측하는 1단계 방식을 채택했습니다.

- YOLO는 classification 과 Localization을 동시에 하여 실시간으로 객체의 검출이 가능
- YOLO는 특징적으로 **각 grid cell에서 여러 개의 bounding box를 예측**하며, 각 bounding box에 대한 객체 존재 확률과 클래스 확률을 함께 출력함
- 하지만, 하나의 객체에 대해 여러 개의 overlapping된 bounding box가 예측되는 경우가 많으며, 이를 그대로 사용하면 **중복된 탐지 결과가 많아지고, 최종 출력이 혼란스러워지는 문제**가 발생함
- 이를 해결하기 위해 Non-Maximum Suppression(NMS)을 적용하여 가장 신뢰도가 높은 bounding box만 남기고 중복된 box를 제거함.
- 옴로 모델의 단점
 - 작은 객체 검출: 그리드 기반 접근법으로 인해 작은 객체나 밀집된 객체 검출에 어려움이 있습니다.
 - 정확한 경계 상자: 단일 패스 처리로 인해 경계 상자의 정밀도가 2단계 방법보다 낮을 수 있습니다.
 - 클래스 불균형: 실시간 처리를 위한 단순화로 인해 희귀 클래스 검출 성능이 저하될 수 있습니다.

YOLO 객체 탐지 구조



(1) 입력 그리드 ($S \times S$ Grid)

- 이미지를 $S \times S$ 그리드로 나눕니다.
- 객체의 **중심점(center)** 이 위치한 그리드 셀이 해당 객체 검출을 담당합니다.
- 즉, 각 셀은 “내 영역에 객체가 있나?”를 판단합니다.

(2) 각 셀의 예측값

- 바운딩 박스 (Bounding Boxes)
 - 각 셀은 **B개의 바운딩 박스**를 예측합니다.
 - 각 박스는 다음 5가지 값을 가집니다:
 - (x, y): 박스 중심 좌표 (셀 기준 상대 위치)
 - (w, h): 박스의 너비와 높이 (이미지 크기 대비 비율)
 - confidence: 객체가 있을 확률 + 박스 정확도
- 클래스 확률 (Class Probabilities)
 - 각 셀은 **C개의 클래스 확률**을 출력합니다.
 - 확률값 = $P(\text{Class} \mid \text{Object})$
 - 즉, “객체가 있다면 이 클래스일 확률”.

(3) 신뢰도 점수 (Confidence Score)

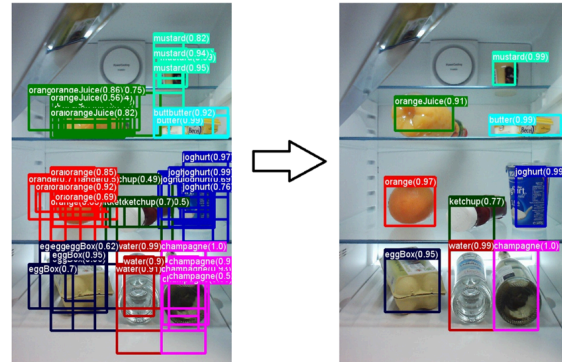
- 각 바운딩 박스의 최종 점수는

$$\text{Confidence} = P(\text{Object}) \times \text{IOU}(\text{pred}, \text{truth})$$

- 여기서 IOU는 예측 박스와 실제 박스의 겹치는 정도.
- 따라서 객체가 있을 확률과 위치 정확도를 동시에 반영합니다.

(4) 최종 탐지 (Final Detections)

- 모든 셀과 앵커박스에서 박스들을 모읍니다.
- 신뢰도 점수가 낮은 박스는 제거합니다.
- **NMS (Non-Maximum Suppression)** 적용 → 중복된 박스 제거. (우측이미지)
- 남은 박스가 최종 탐지 결과로 출력됩니다.



NMS(Non-Maximum Suppression)는 객체 탐지 모델이 예측한 여러 개의 중복된 바운딩 박스 중에서 가장 정확한 하나만 남기는 후처리 과정입니다. 모델은 하나의 객체에 대해 여러 개의 바운딩 박스를 예측할 수 있는데, NMS는 이 문제를 해결해줍니다.

1. **신뢰도 정렬:** 모델이 예측한 모든 바운딩 박스를 **신뢰도 점수**가 높은 순서대로 정렬합니다.
2. **최고 신뢰도 박스 선택:** 가장 높은 신뢰도 점수를 가진 박스를 최종 결과로 선택합니다.
3. **겹치는 박스 제거:** 선택된 박스와 IoU(Intersection over Union)가 일정 임계값(threshold) 이상으로 겹치는(overlap) 다른 모든 박스들을 제거합니다.
IoU는 두 박스가 얼마나 겹치는지를 나타내는 척도입니다.
4. **반복:** 남은 박스들 중에서 가장 높은 신뢰도를 가진 박스를 다시 선택하고, 이 과정을 모든 박스가 처리될 때까지 반복합니다.

이 과정을 통해 NMS는 한 객체에 대해 여러 개의 예측 박스가 생성되는 것을 방지하고, 가장 신뢰도가 높은 단 하나의 박스만 남겨 객체 탐지 결과를 깔끔하게 만들어 줍니다.

3 YOLO 네트워크 & 손실함수

- YOLOv1 아키텍처

초기 YOLO는 GoogLeNet에서 영감을 받은 24개의 컨볼루션 레이어와 2개의 완전연결층으로 구성되었습니다. 입력 이미지는 448×448로 조정되고, 최종 출력은 7×7×30 텐서입니다 (7×7 그리드, 각 셀당 2개 박스, 20개 클래스).

- 주요 레이어 구성

네트워크는 크게 세 부분으로 나뉩니다. 특징 추출을 위한 컨볼루션 레이어들이 앞부분에 위치하고, 중간에는 차원 축소를 위한 풀링 레이어들이 있으며, 마지막에는 예측을 위한 완전연결층이 위치합니다. 활성화 함수로는 Leaky ReLU를 사용하여 음수 영역에서도 작은 기울기를 유지합니다.

- 손실 함수 설계

YOLO의 손실 함수는 세 가지 요소를 결합하고 (다중 작업 손실), λ_{coord} , λ_{noobj} 로 가중치 균형을 설정합니다.

손실 유형	설명
좌표 손실 (Localization Loss)	바운딩 박스의 x, y 중심 좌표와 너비(w), 높이(h)의 예측 오차를 측정합니다. 작은 박스의 편차가 큰 박스보다 더 중요하므로, 너비와 높이에는 제곱근을 적용합니다.
신뢰도 손실 (Confidence Loss)	객체가 있는 셀과 없는 셀의 신뢰도 예측 오차를 계산합니다. 대부분의 셀에는 객체가 없으므로, 배경에 대한 가중치를 줄여 균형을 맞춥니다.
분류 손실 (Classification Loss)	객체가 있는 셀에 대해서만 클래스 예측 오차를 계산합니다.

📌 λ_{coord} , λ_{noobj}

YOLO 논문(특히 YOLOv1)에서 기본적으로 $\lambda_{coord} = 5$, $\lambda_{noobj} = 0.5$ 로 설정해 두었고, 이후 YOLOv2~v3에서도 같은 값 사용

- λ_{coord}
 - YOLO의 손실 함수(loss function) 안에서 "좌표 오차 부분에 곱하는 가중치"
 - 좌표 오차(위치, 크기)를 더 중요하게 반영하겠다는 뜻.
 - 단순히 "사람 있음"을 맞추는 것보다 **사람의 위치와 크기**가 정확해야 의미가 필요할때,
좌표 손실에 5배 가중치를 줍니다.
 - λ_{noobj}
 - YOLO 손실 함수에서 "**객체가 없는 셀(no object cell)**"의 예측 오차에 **곱해지는 가중치**
 - 즉, 배경(background)인데 객체(confidence score > 0)라고 잘못 예측한 경우에 주는 패널티의 세기 조절값
 - 객체가 없는데 "있다"라고 잘못 예측한 경우를 너무 과도하게 벌주지 않도록 하는 값.
 - 실제 이미지에는 배경이 훨씬 많으므로, 이 값을 너무 크게 잡으면 모델이 "**전부 배경**" 만 예측해버리는 문제가 생김.
-
- 정리
 - $\lambda_{coord} = 5$
위치/크기 예측의 중요성을 감안하여, 예측 오류에 대한 패널티를 크게 부여
 - $\lambda_{noobj} = 0.5$
배경 클래스에서의 오탐지(false positives)에 대해서는 패널티를 약하게 부여

즉, YOLO의 학습 목표는 "**정확한 위치 예측**"에 무게를 두면서, 배경 실수는 적당히 무시" 하는 구조

○ 성능 평가 지표

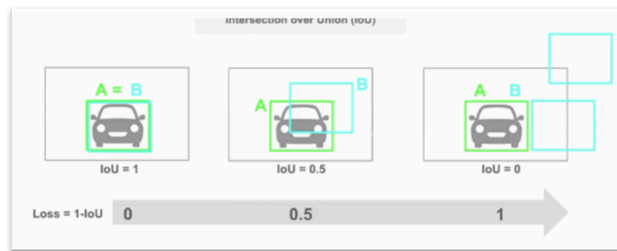
- mAP (mean Average Precision)

각 클래스별 AP를 계산한 후 평균을 구합니다.

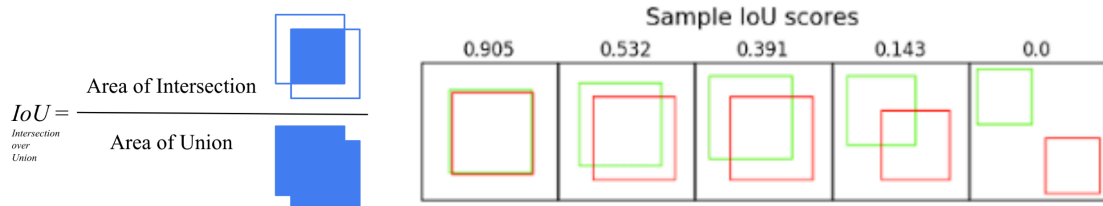
IoU 임계값에 따라 mAP@0.5, mAP@0.75 등으로 표기합니다.

IoU란? ※ Intersection over Union

- 두 개의 영역 간에 겹치는 정도를 측정하는 지표



$$IoU = \frac{\text{교집합 면적}}{\text{합집합 면적}}$$



- 8.2 FPS (Frames Per Second)

실시간 처리 능력을 나타내는 지표로, YOLO의 주요 장점 중 하나입니다. 일반적으로 30 FPS 이상이면 실시간 처리가 가능하다고 봅니다.

4 YOLO의 진화

- YOLOv2 (YOLO9000)

YOLOv2는 배치 정규화를 도입하고, 고해상도 분류기를 사용하며, 앵커 박스 개념을 추가했습니다. Darknet-19라는 새로운 백본 네트워크를 사용하여 속도와 정확도를 개선했습니다.

- YOLOv3

다중 스케일 예측을 도입하여 작은 객체 검출 성능을 크게 향상시켰습니다.

FPN(Feature Pyramid Network)과 유사한 구조를 채택하고, Darknet-53 백본을 사용했습니다.

- YOLOv4와 YOLOv5

YOLOv4는 CSPDarknet53 백본과 다양한 최신 기법들(Mish 활성화, DropBlock 정규화 등)을 통합했습니다. YOLOv5는 PyTorch로 구현되어 사용성이 크게 개선되었고, 자동 앵커 박스 최적화 등의 기능을 추가했습니다.

- **최신 버전들 (YOLOv8, YOLOv9)**

최신 버전들은 Transformer 기반 구조를 일부 도입하고, NAS(Neural Architecture Search)를 활용한 최적화, 더 효율적인 학습 전략 등을 통해 지속적으로 성능을 향상시키고 있습니다.

옐로 버전별 성능

YOLOv1 (2015): 45 FPS, 63.4% mAP

YOLOv2 (2016): 67 FPS, 76.8% mAP

YOLOv3 (2018): 65 FPS, 82.0% mAP

YOLOv4 (2020): 65 FPS, 85.4% mAP

YOLOv5 (2020): 140 FPS, 88.2% mAP

YOLOv8 (2023): 200+ FPS, 92.1% mAP

기본적인 옐로(YOLO) 모델은 **사각형 형태의 바운딩 박스**만 검출합니다. 각 객체의 위치를 x,y,width,height 네 가지 값으로 나타내기 때문에 다각형을 직접 인식하지는 못합니다.

하지만 최근 버전인 **YOLOv8** 모델은 인스턴스 세그멘테이션(Instance Segmentation) 기능을 추가하여 **다각형 모양의 객체**도 검출할 수 있게 되었습니다.

- 바운딩 박스와 인스턴스 세그멘테이션의 차이

- **바운딩 박스 (Bounding Box):** 객체를 포함하는 가장 작은 직사각형을 그립니다. 빠르고 효율적이지만, 객체의 정확한 윤곽을 표현하지는 못합니다.
- **인스턴스 세그멘테이션 (Instance Segmentation):** 객체의 정확한 픽셀 단위 영역을 찾아내어 마스크(Mask)를 만듭니다. 이 마스크는 객체의 실제 모양을 따라가는 다각형과 같은 형태를 띄게 됩니다.

따라서 옐로의 최신 버전을 활용하면 객체를 사각형이 아닌 더 정교한 다각형 형태로 탐지하는 것이 가능합니다.

